MMA3001 Project Report

Ellena Glenk 34957189
October 2026

The Engineering Problem

This engineering project sought to obtain an average structure of an apple tree to be used in the development of farming machinery. During operations, machinery must interact with trees of all different sizes and shapes, and may have to interact with trees at different times of the year. For these machines to complete their job well (eg, for maintenance, harvesting, or orchard navigation), they must accommodate a range of tree structures that may be present in the particular orchard they are used in. Given that apple trees may have varying trunk widths, branching locations, spread and symmetry, a generalised average tree makes a good starting point for the design of machinery. Computational methods of analysis for this task will be more efficient at analysing these trees objectively, rather than manually measuring each individual tree.



Figure 1. An example of the RGB, depth and ground truth images available for each tree in the dataset.

The project aims to take visual image data and recreate an average tree structure. The data available for use included RGB images, depth encoded images, and manually annotated tree structures for each individual tree in the apple tree orchard (as seen in figure 1). To create the model, images needed to be analysed to observe irregular tree structures and inconsistent tree visibility. The chosen numerical method to complete this task needed to represent the non-linear structure of the apple trees, while minimising any excessive computation required.

Using numerical methods and machine learning for this problem can replace time consuming manual measurements which may give incorrect or inconsistent models. A model built with these techniques could recognise trends in larger datasets, automating identification of tree structures and supporting early design decisions in machine manufacturing. However, a model of the apple trees present in the dataset cannot be extrapolated as a model for every tree that potential farm machinery may encounter.

The computational solution produced by this project could be used to develop farm machinery, certainly for this orchard but potentially for other apple (or other) orchards as well. Once the computational process has been shown to work for these apple trees, it could be developed over time to reflect the growth of the trees, and potentially assist in agricultural machinery design for more established apple trees in future.

The initial goal for this project was to construct the average 3D shape of an apple tree using both RGB and depth data. However, with depth image analysis and a lack of information surrounding the metadata of the depth images, a conversion from greyscale images to a real world depth value was not plausible. Furthermore, images of the same tree in different seasons were taken at different locations, creating uncertainty in the true depth and 3D shape

of trees which could be modelled. Early 3D model constructions produced noisy, irregular models, and thus the decision was made to create a 2D model with greater certainty than a potentially inaccurate 3D model.

Inputs and Outputs

The ZIP file of images used as input data included RGB and depth images for each tree in the orchard, identified by a row number and position number. For each tree, there was an RGB and depth image for both a summer and winter season, and a ground truth binary image showing the shape of the trunk and main branches for the summer trees. Python functions were developed to match image types to each individual tree. Unmatched trees were not included in the dataset.



Figure 2. Visual representation of croping vs resizing techniques for image scaling.

The input images had to be prepared to ensure models could use them comparatively to get a quality result. Original images and ground truth images had different dimensions, so both a scaling and cropping method were tested to identify which method was more accurate. Cropping risked removing branches of the tree which were needed to create the tree model, so resizing to 256 x 256 pixels worked best (this is shown in figure 2). All inputs were checked for missing data before being used as training data. Some depth images contained zeros, which was uncertain as to the real-world result of this. This data was either treated as unavailable for analysis, or interpolated if required for analysis.

The top level outputs of this project included geometric tree measurements (including base width, branching position, branch spread and symmetry), statistical average tree centrelines (from ground-truth analysis), model performance metrics (such as MAE (mean average error), RMSE (root mean squared error), Dice and IoU), neural network (U-net) probability maps and binary masks, and a final average 2D probability map and corresponding binary tree model.

Geometric tree measurements and tree centrelines were used for the initial linear interpolation and decision tree models of average apple tree shape. Model performance metrics were used to numerically compare models to determine which was most successful and applicable to our engineering problem. Probability maps and masks were used to come to the final output of the average apple tree model (as shown in figure 3).

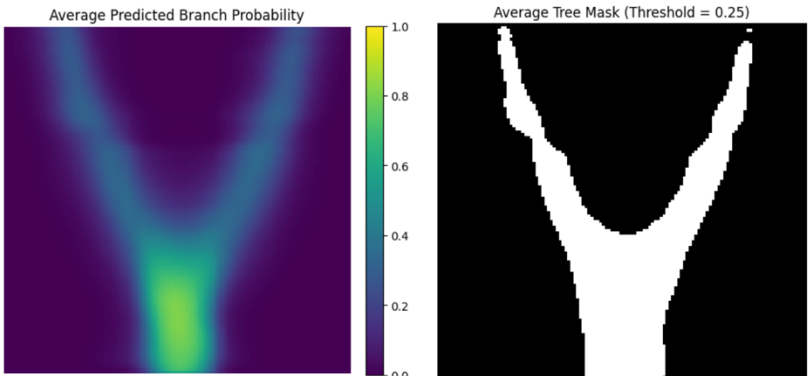


Figure 3. Probability map and binary mask of average apple tree, found as the conclusion from this numerical project.

Computational Solution

Before any numerical or machine learning method was used, data was separated into a training and validation dataset by tree ID (row and number). A fixed random seed was used to separate trees into an 80/20 training/validation split, with both summer and winter images to be used. This would prevent models seeing the same tree twice, producing overly-optimistic tree models.

Computational methods were tried and tested, and improved or technique changed where required. The following information is presented in chronological order, with the logic behind decisions made outlined as they were made.

Numerical Interpolation of Depth Data

Some depth images were found to have some pixels assigned a value of 0. Without information regarding how the depth images were taken, methods were established for interpolating this data for use in the original 3D modelling objective.

The first method tested was nearest neighbour interpolation, which estimated the value of a pixel from its nearest known pixel's value. This method is relatively simple and computationally inexpensive, but may produce abrupt changes between regions of the image. This method was used as a baseline as it required little computation and provided a straightforward comparison.

The second method tested was linear interpolation, which used nearby known points and a piecewise linear function to estimate depth values. This method expects to create smoother variations in depth than the nearest neighbour method, and reduce abrupt discontinuities. This method required suitable neighbouring measurements, and could become unreliable over large gaps.

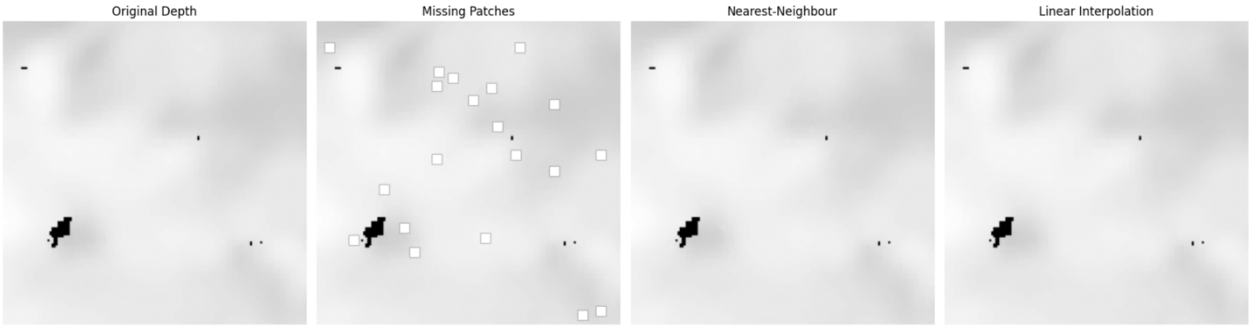


Figure 4. Visual of depth data interpolation via nearest neighbour interpolation and linear interpolation.

Both methods were tested within the training data set against a small amount of withheld pixels, as shown in figure 4. The RMSE for each method was calculated and compared; the nearest neighbour method had RMSE of 0.6653, while the linear interpolation method had RMSE of 0.4088. Both methods were then further tested on larger missing patches of pixel information, where real depth images contained some spatial gaps of varying size. Linear interpolation was still found to be the stronger model under these circumstances. It is important to note that a lower RMSE in these methods could not prove that an interpolated depth value was true in the real world, which contributed to the eventual decision to create a 2D instead of 3D model.

Ground Truth Geometry Extraction

The ground truth masks were analysed to determine physical features of each individual tree, such as trunk width, annotated area, branch position and horizontal branch spread. Functions were developed to identify the approximate branching point based on changes in the horizontal width of the structure, and the left and right branches were analysed separately due to the asymmetry of most trees. The data from this analysis was compared and visualised across the dataset (as seen in figure 5).

Initially, numerical target values were needed to train regression models, and geometric measurements provided the means to compare treed quantitatively. We were also able to investigate whether a simple parameter based model could accurately build an average tree model. However, reducing trees to a small number of measurements meant important information about branch curvature and local shape were lost. This limitation became important as initial regression models were interpreted.

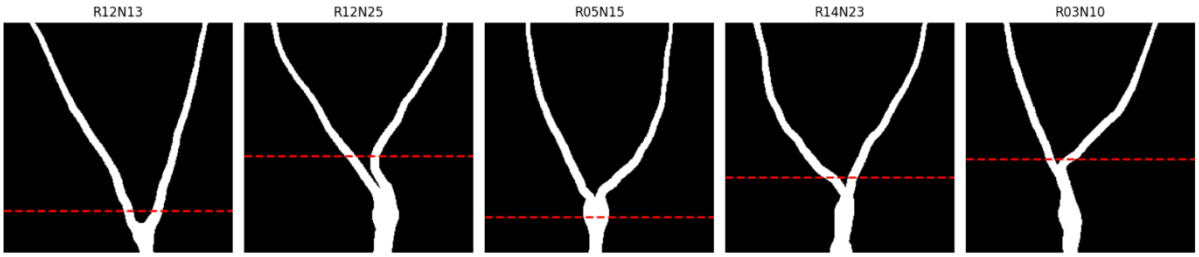


Figure 5. Ground truth masks from five tree examples displaying the predicted branching point based on trunk width.

Machine Learning Regression Models

Three approaches for the initial machine learning models were used:

A mean-baseline model (Dummy Regressor) was used to predict an average geometrical tree value for each of the geometric target characteristics. This model did not use image features to distinguish between individual trees, rather a singular average tree was used as a reference for determining if more complex methods could better predict a tree's shape. This baseline model was chosen as it is inexpensive to train and predict.

A decision tree model (Random Forest Regression) took combined predictions from multiple decision trees to predict a tree's structure. This model was selected as it can model non-linear relationships between image features and geometrical measurements. This particular decision tree used 200 trees with a maximum depth of 12 and minimum leaf size of 3. While this model improved some geometric measurements (3 out of 6), it did not consistently outperform the mean-baseline model.

Finally, a neural network (Multilayer Perceptron Regression) was used to investigate if a feedforward neural network could learn more complex relationships between image features and tree geometry. Numerical inputs were standardised to support more stable optimisation, using 64 hidden layers and 32 neurons. Early stopping was used to

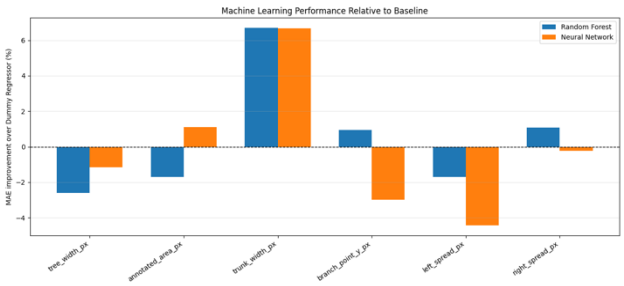


Figure 6. Visualisation of RMSE for each target feature for both the RandomForest and Neural Network models as an improvement on the mean-baseline model.

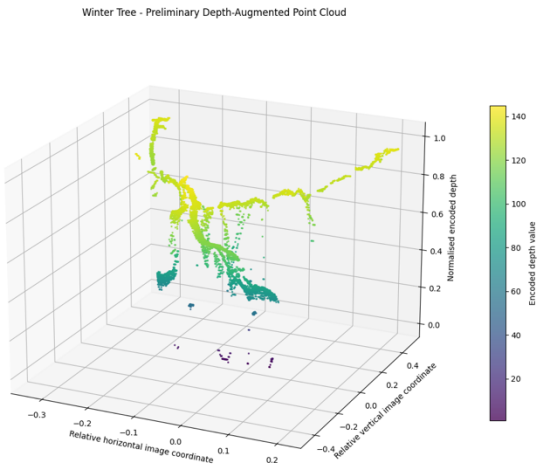


Figure 7. 3D depth imaging of a tested winter tree.

prevent overtraining. However, improvement on performance was limited for several geometric targets.

Given the decision tree model and the neural network both struggled to perform significantly better than the mean-baseline model (as seen in figure 6), more analysis was conducted on the input data as an attempt to improve our models. Global image features summarised the RBG and depth data of an entire image, so the input images were divided into spatial regions to calculate features within those regions. This improved some target predictions but worsened others, and the increased number of input features introduced additional complexity without substantial performance improvement. Given that manually manipulated spatial features were insufficient in improving geometry prediction, a convolutional neural network capable of learning spatial features from the images was used.

Statistical Average 2D Tree Model

Given the lack of improvement of more complex models, we returned to a simpler method of analysis: a statistical 2D average tree model. Ground truth images had their left and right branches separated and their trunk and branching locations found. The tree structures were aligned to a common reference to reduce image framing differences. Sampled centreline coordinates were used to represent curved branches, and piecewise linear interpolation was used to estimate branch positions at common relative heights above branching location. The average left and right branches were then calculated across training trees.

This method was used as curved tree branches could be modelled and accounted for during training. Data from trees with different branch lengths could be compared more easily, and averaging interpolated centrelines produced a branch shape that could preserve more information than just width and height measurements. The method is also more computationally straight forward and results are easy to interpret. However, the model also only predicted the average shape from above the branching point: information regarding relative branch height, trunk height and thickness and other geometrical predictions were not found (as shown in figure 7). Using this model to design agricultural machinery may not have considered enough variation to accommodate many tree shapes.

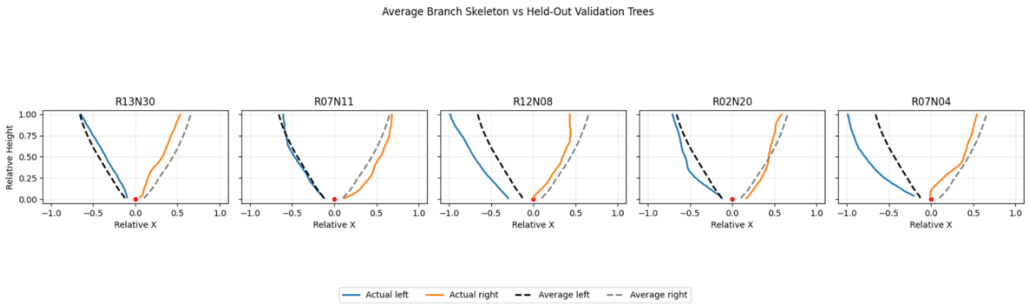


Figure 8. Visual representation of statistical model average compared to validation tree examples.

U-Net Neural Network

A U-Net neural network was chosen as the final model for processing this apple tree image data. Previously used models reduced each tree to its numerical measurements, which limited its ability to represent complete trunk and branch shapes. The U-Net model provided pixel-level predictions, rather than predictions based on geometry values alone. The model is able to learn spatial patterns present in the images such as edges, branch locations and branch structure. It can combine broader contextual information with finer image details, and allows more information from early layers to have a greater effect on the final model. This made the U-Net model more suitable for the task of producing a complete 2D tree model.

The model used a convolutional neural network with 487441 trainable parameters and a maximum epoch of 30 to control training. It's training loss was measured using a combination of binary cross-entropy and dice loss to balance a measure of individual pixel classification with complete annotated tree structure overlap. This helps reduce the noise from the background and leaves of images, as the model we are attempting to develop only takes up a small amount of the pixels in the image. It was noticed that the model could quickly learn the Y shaped structure of the tree but sometimes missed thin branches, or disconnected branches entirely. The probability threshold was thus optimised to include as much of the branch structure as possible: eventually reaching 0.25.

This threshold was then applied to the validation dataset without further training (this is not retraining the network, rather changing how probability maps were converted into binary masks). This gave a larger model, but one more appropriate to design farm machinery with (as seen in figure 8). It is important to note that this model with a lower threshold may have caused some false-positive branch pixels, and also results in a trade off between identifying genuine branches and interpreting some background as part of the tree instead.

While this method was identified as the best method used in this project for the purpose of designing farm machinery for use amongst these apple trees (where a shape including more potential areas for branches of the tree is more important than missing that they are there), this method does not indicate accurate tree shape or representation.

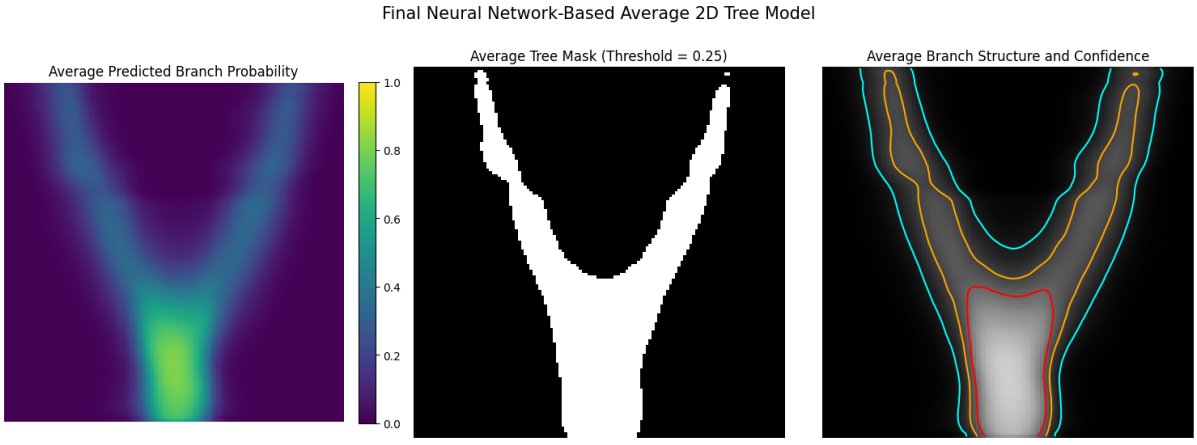


Figure 9. Images produced as final neural network based 2D apple tree model.

Alternative Solutions

The alternative solutions for this project have been detailed as above in the chronological order they were attempted. A summary of their advantages, limitations and ultimate decisions have been summarised below.

Section of project	Approach	Advantages	Limitations	Decision	
Depth interpolation for missing values	Nearest Neighbour	Simple, fast	Abrupt changes, higher RMSE	Not used for depth interpolation	Difficulty in exploring depth data ultimately led to a more accurate 2D model being preferred over a potentially incorrect 3D model.
	Linear Interpolation	Lower RMSE	Unreliable when information becomes insufficient	Used for depth interpolation when depth values missing	
2D Tree Model	Mean-baseline	Simple and interpretable	Fail to capture differences between trees	Used as regression baseline	
	Random Forest Regression	Models non-linear relationships	Inconsistent improvement from baseline	Not selected	
	MLP Regression	Learns non-linear feature relationships	Limited spatial understanding	Not selected	
	Statistical 2D model	Curved branch representation	Relies on centrelines from masks	Not selected	
	U-Net segmentation	Predicts spatially detailed masks	Computationally expensive, moderate accuracy	Used as final 2D average tree model	

The U-Net model was ultimately chosen due to its application to the engineering problem. While other methods were more computationally more efficient, some providing similar RMSE values, some methods created models that may have not resulted in the best informed agricultural machinery design process. For example, the statistical 2D model created a fairly robust, symmetrical, thin branched average model of the trees in the orchard, with almost no consideration of variation in shape, branch length or asymmetry. If this were to be used to design machinery, the designer may assume all the trees look incredibly similar, without leaving much room to account for the wide range of trees actually seen in the orchard. While the U-Net model is more expensive computationally, its probability map and ability to have thresholds edited means the machine designer will be aware of large variations in tree shape and size, preventing the accidental damage to trees and apple produce while the machine is in use.

Validation

The initial step before any models were trained was to separate training and validation data. For all models, experimental or chosen, 449 trees and all their corresponding images were selected randomly as training data, and 113 trees and their images were kept as validation data. This 80/20 split allowed for models to be trained and validated against the same data, meaning their resulting errors and models were easily comparable. Splitting the data by tree and not by images also meant the model never saw the same tree twice, as there are multiple images present across seasons and in different formats that we want to avoid our model to overfit.

Validation of Depth Interpolation

While experimenting with depth data and pixel depth interpolation, some known depth pixels within the training dataset were temporarily withheld. Interpolation methods attempted to reconstruct these values, which were then compared to the known withheld values. This was further tested by withholding larger patches of pixels and testing the predicted value against the known depth value. RMSE was used to compare these methods as it penalises larger errors more harshly, and we needed to ensure depth data was accurate for model development. These experiments were interpolating encoded depth values, rather than actual measured distances, which is why the 3D element of this model was eventually removed.

Validation of Extracted Geometry

Geometry extracted using python functions was compared to the ground truth masks provided with each tree in the training dataset. This was then compared to the validation dataset to assess whether branching points and centrelines followed a pattern in the ground truth masks. This was imperfect for most irregular trees, which motivated using statistical averages.

Validation of Statistical Average Model

The average branch centreline was evaluated against the validation dataset, and the mean branch position errors were compared to a simpler straight branch baseline. MAE and RMSE were used in relative horizontal image coordinates, and thus indicated that the statistical curved line model provided a more representative geometric approximation. The success of this model was limited by the need for manual alignment and the lack of general information that a tree model may need (trunk width variation, branch spacing variation, etc).

Validation of U-Net Neural Network

The probability cloud of the neural network was evaluated against the 113 validation ground truth images, and measured numerically using a mean dice and IoU (intersection over union) score. Both scores are a measure of the overlap of each predicted tree model and the ground truth image. The more overlap, the higher the score.

$$Dice = \frac{2 \times overlap}{actual\ pixels\ + predicted\ pixels} \qquad IoU = \frac{overlap}{actual + predicted - overlap}$$

The Dice score was 0.3796, and the IoU score was 0.2389. These scores were used to quantify the accuracy of the model as pixel number alone would not justify the model's success. However, these two scores indicate the models still have substantial error associated with them. Tree structure has been partially learnt, fine branches and branch shape still requires improvement. A visual image of a Y shape tree is not sufficient proof of an accurate model.

Optimisation and Performance

Improvements to Numerical Interpolation
Linear and Nearest neighbour interpolations were compared against known pixel values and the RMSE for each was calculated. Linear interpolation as eventually selected for depth reconstruction calculations as its RMSE was significantly lower. This was further tested by increasing the size of the missing pixel area, but could be further tested by cropping the available known image and testing again to validate accuracy.

Improvements to Machine Learning Features
Originally, machine learning inputs used the global RGB data for analysis and training. However, it was found that global features contained limited information about spatial branch positions and distribution. Spatial feature extraction was tested in an attempt to retain more information about branch location, though results were mixed. This indicated that feature count alone could not guarantee better accuracy. This could be further improved by putting more emphasis on winter images during training, where RGB distinctions are greater and the model may learn structures and differences between branches and background more efficiently.

Improvements to Neural Network
Images were reduced to 128 x 128 pixels to reduce the computations required for each image analysis. The U-Net selected was designed to be compact enough without losing too much image and prediction quality. Early stopping was also implemented to prevent overfitting and unnecessary training. For greater improvement, the image size constraint could be further explored: does reducing the pixel size of the training images this much result in the loss of fine-branched structures, ultimately impacting the accuracy of our model?

A separate development was also used to monitor training and to select the best probability threshold. A probability threshold of 0.5 was originally chosen but was found to ignore some narrow tree structures, so a threshold of 0.25 was selected instead. This improved the dice score of the final model. The threshold could be explored in more detail in future to determine its effect on data analysis.

RGB vs Depth Sensitivity
The neural network's reliance on both RGB images and depth images was investigated by replacing one input modality at a time with constant training-mean values. The results of these was compared to the original RGB-depth predictions. It was found that retaining depth images while neutralising RGB images more closely resembled the original output. This suggests that depth imaged had a greater influence on the trained model than RGB images did. This could be explored further in future analysis, and the reliance on each image potentially altered to improve the tree model.

AI Use and Reflection

OpenAI's ChatGPT was used to assist in the initial understanding of the project, the first analysis of the data, and for a substantial amount of the creation of the Python code in the Google Colab Jupyter notebook. ChatGPT was used to outline the topics and points of interest to be discussed in this project report, though no text in this PDF report was generated by AI or copied from an AI: all writing has been completed by the author Ellena Glenk. Images in this report are either taken from the dataset provided by Monash University or as

graphs and images generated from code in the Colab notebook, which was mostly generated by ChatGPT.

ChatGPT was used for the initial description of what the project entailed and what it required of me, as it initially felt overwhelming. I uploaded a detailed prompt and the project documentation I had, and after reading its output thoroughly, I was able to gain a better understanding of the project, and begin to write a thorough prompt to gain its assistance in writing code for the Colab notebook.

ChatGPT generated all blocks of code used in the project (aside from some GitHub commit functions called), along with detailed explanations of what each block was doing. As I am not confident with coding, but I understand the numerical methods I wished to use for this project, I asked AI to help me write the code to complete the functions and analysis I intended. These code blocks included comments, which remain with some of my own edits in the current Colab document. Once I had read all ChatGPT outputs, I asked it more questions to gain clarity or change its output, before copying the code into my Colab notebook. Where needed, I independently verified its output using resources online, the MMA3001 unit notes, or classmates. The direction of the project, including the initial engineering problem, ensuring analysis types trialled were from MMA3001 covered topics, making decisions to remove the 3D aspect of the project, to use a neural network, and to further train some models and discard others were decisions made by myself, not ChatGPT. The structure, organisation and heading titles in the Colab notebook are all completed without AI use.

After all code blocks had been completed, I read through the entire document to ensure I was happy with the process taken to get to my final result and ensure the document had a logical flow. I edited many code block by removing unnecessary or repeated code, rearranging blocks where required, and adding detailed headings to assist the reader's understanding. Everything submitted in this project has been reviewed by Ellena Glenk.

I found that ChatGPT often would want to continue optimising and analysing a task when it seemed to hit a dead end. In some early model creations, the AI would wish to continue delving deeper into refining the model even when its results consistently showed poor tree models. I would sometimes suggest taking a different direction (eg. using a neural network) and it would always agree with me, which made it difficult to determine what was actually the most beneficial thing to do. I often had to point it to a different set of data, or a different output, to get the result and code block that I wanted.

However, ChatGPT more often than not provided helpful information for me to complete this project. The AI was able to suggest actual Python functions to use to obtain the numerical method I wanted, and was able to generate incredible images to assist with my visualisation of the project. It often increased my understanding of the project itself, the techniques I was using to solve it, and when I had conceptual understanding questions it was able to give me detailed, useful explanations. I also learnt, most importantly, how to use AI to get the response I wanted. If I had questions I thought it could not help me with, I soon found myself able to complete a task with a great understanding that I previously felt too massive to undertake. After this project, I am confident I could use AI in the future to assist me in numerical analysis and machine learning problems I may encounter.

The link to the entire conversation I had with ChatGPT for this project can be found here: https://chatgpt.com/share/6adf4517-b544-83ec-9e09-f7e282487fae